Weekly Student Progress Performa

Team Id	CS26084	**Week no / Date**	Week 2 27 July 2026
Project Title	SentinelCore: A Network Detection and Incident Response Platform		
Members Present	Om Rashiya (24cs084), Yash Patel (24cs082)		
Mentor	Akshita Kadam		

WEEK 2 • Threat Model and Secure Architecture

Section	Details to be filled by the team
Planned Tasks (from last week)	Create a threat model, define functional/non-functional requirements, and design a safe deployment architecture.
Tasks Completed This Week	Documented trust boundaries for the browser, API, database, Redis queue, sensor, and privileged network helper. Defined RBAC roles, auditability, retention, least privilege, local deployment, and recoverable operations as core requirements.
Commits Made	Total: 0 commits (architecture and research week). Outputs were the threat model, data-flow plan, and milestone breakdown used for later implementation.
Issues Created / Closed	Created: Raw packet capture and firewall operations require privileges that the web API must not receive. Closed: Introduced a narrow Unix-socket helper boundary so the API can remain unprivileged.
Milestone Status	~10% complete — on track. Architecture and security constraints were established before coding.
Blockers / Challenges Faced	Balancing useful network visibility with safe privilege separation. The design isolates raw-socket operations and exposes only allow-listed helper commands.
Plan for Next Week	Design the database entities, API contracts, Docker topology, and initial repository structure.

Research and Technical Study

Research objective. Create a threat model, define functional/non-functional requirements, and design a safe deployment architecture.

Study completed. Studied container capability controls, Unix-domain socket permissions, secure refresh-token patterns, PostgreSQL transaction guarantees, and Redis-backed work coordination. The design adopted defence in depth: capability allowlists, unprivileged API execution, append-only audit records, and explicit role checks.

Trust-boundary and control mapping

Boundary	Risk considered	Chosen project control
Browser → API	Unauthorized actions	JWT session plus backend RBAC
API → helper	Privilege escalation	Typed allow-listed Unix-socket operations
Worker → database	Replay or event loss	Durable writes and checkpoints
Operator → firewall	Self-block or forgotten block	Protected IPs and mandatory TTL

Decision carried into the project

Documented trust boundaries for the browser, API, database, Redis queue, sensor, and privileged network helper. Defined RBAC roles, auditability, retention, least privilege, local deployment, and recoverable operations as core requirements.

Implementation Record and Website Evidence

Trust boundaries

Separated the browser, reverse proxy, React application, FastAPI service, PostgreSQL, Redis, Suricata, event worker, and privileged helper. Each boundary was assigned only the communication path needed by the product.

Privilege model

Nmap, Suricata control, and firewall changes were placed behind an allow-listed helper reached through a Unix socket. The web API was designed to run without raw-socket or firewall capabilities.

Access model

Defined viewer, analyst, and admin responsibilities. Read-only visibility, investigation work, and high-impact administrative actions would be enforced by backend role checks and reflected in role-aware navigation.

Recovery and evidence

Auditability, bounded retention, explicit action history, and recoverable operations were made architectural requirements. These choices support the website promise of visible evidence and controlled response.

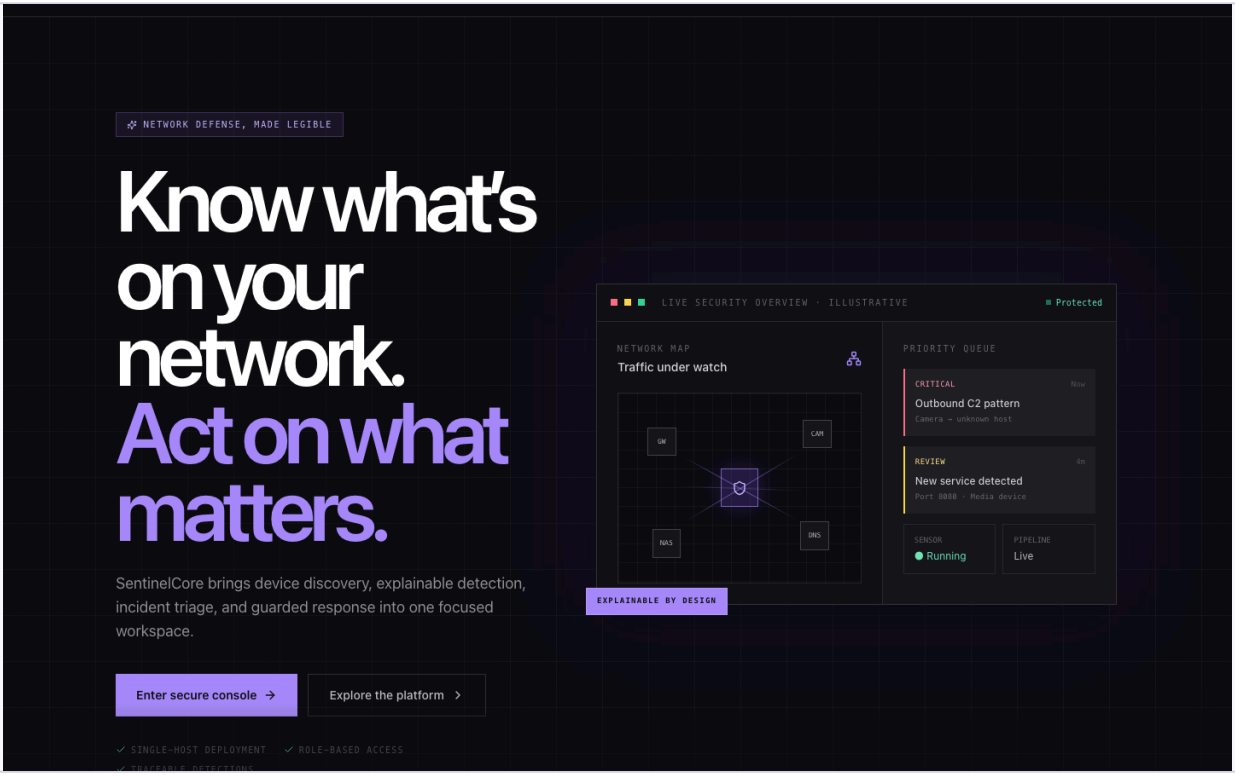


Figure W2.1 — Project-owned SentinelCore interface mapped specifically to Week 2.

Evidence interpretation. The public landing page expresses the planned product flow in user language: know the network, explain detections, investigate evidence, and respond with guardrails. Its visible controls, labels, status fields, and navigation provide direct evidence for the work described on this page.

Repository and Architecture Evidence

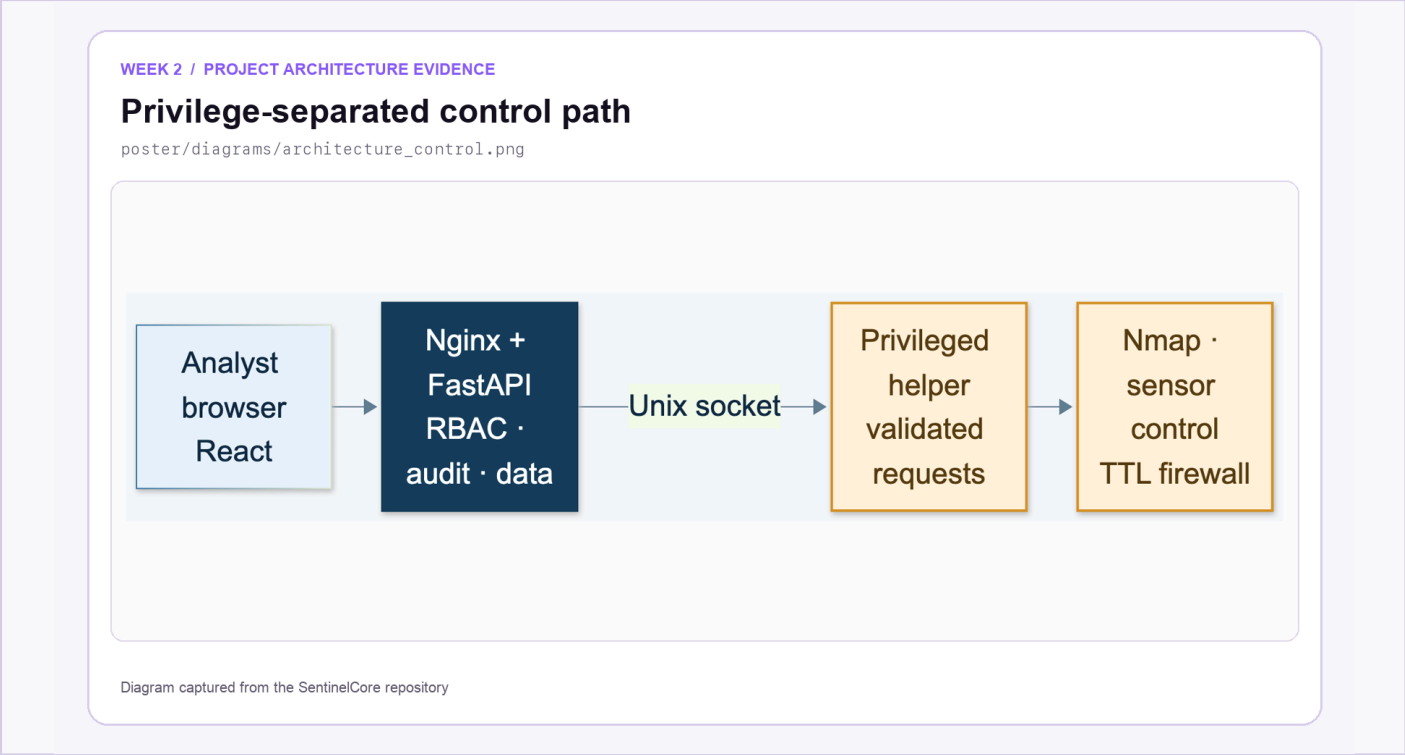


Figure W2.2 — Privilege-separated control path. This is a separate internal artifact, not a copy of the website image.

Repository files used for traceability

- reports/docs/threat-model.md
- docker-compose.yml
- README.md — Architecture, roles, and security design

Verification

The architecture record was cross-checked against the final Compose services and the documented privilege split; no cloud service or unimplemented external dependency was added.

Traceability conclusion. Privilege-separated control path directly supports the Threat Model and Secure Architecture milestone. The architecture record was cross-checked against the final Compose services and the documented privilege split; no cloud service or unimplemented external dependency was added. The artifact, source paths, implementation record, and website evidence therefore describe the same weekly scope.

Weekly Closure and Faculty Assessment

Completed outcome

Documented trust boundaries for the browser, API, database, Redis queue, sensor, and privileged network helper. Defined RBAC roles, auditability, retention, least privilege, local deployment, and recoverable operations as core requirements.

Commit record

Total: 0 commits (architecture and research week). Outputs were the threat model, data-flow plan, and milestone breakdown used for later implementation.

Issue and resolution

Created: Raw packet capture and firewall operations require privileges that the web API must not receive. Closed: Introduced a narrow Unix-socket helper boundary so the API can remain unprivileged.

Milestone position

~10% complete — on track. Architecture and security constraints were established before coding.

Challenge handled

Balancing useful network visibility with safe privilege separation. The design isolates raw-socket operations and exposes only allow-listed helper commands.

Next-week hand-off

Design the database entities, API contracts, Docker topology, and initial repository structure.

Repository reference

https://github.com/rashiyaom/SentinelCore

Assessed By (Faculty)	Marks Awarded (5 marks)	Faculty Remarks